

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №5

Реализация межсервисного взаимодействия посредством RabbitMQ

Выполнил:

Ермаков Максим Олегович

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

В рамках домашней работы необходимо подключить и настроить брокер сообщений RabbitMQ или Kafka, а также реализовать межсервисное взаимодействие посредством очереди сообщений. Работа выполнена в папке labs/lab3 на основе микросервисной версии приложения из ЛР2.

- Подключить брокер сообщений RabbitMQ к микросервисной архитектуре.
- Настроить параметры подключения и инфраструктуру брокера через Docker Compose.
- Реализовать публикацию события одним сервисом и обработку события другим сервисом.
- Проверить сценарий через smoke-тест и логи сервисов.

Ход работы

Выбор брокера

Для реализации выбран RabbitMQ. Для текущего проекта достаточно topic-exchange, durable-очереди, подтверждения обработки сообщений и management-интерфейса, который удобно показать на защите.

Параметр	Значение
Контейнер	rabbitmq:3-management
AMQP URL локально	amqp://maindb:maindb@127.0.0.1:5672
AMQP URL внутри Docker	amqp://maindb:maindb@rabbitmq:5672
Management UI	http://127.0.0.1:15672
Exchange	restaurant.events
Тип exchange	topic
Очередь catalog-service	catalog.restaurant-rating

Реализованный событийный сценарий

В ЛР2 рейтинг ресторана после создания или изменения отзыва обновлялся синхронным HTTP-вызовом из review-service в catalog-service. В ДЗ5 этот сценарий переведен на асинхронное взаимодействие через RabbitMQ.

- Пользователь создает или изменяет отзыв через публичный API gateway.
- review-service сохраняет отзыв в review_db и пересчитывает средний рейтинг.
- review-service публикует событие review.rating.recalculated в RabbitMQ.
- RabbitMQ маршрутизирует сообщение в очередь catalog.restaurant-rating.
- catalog-service читает событие и обновляет avgRating и reviewsCount в catalog_db.

Структура события

Routing key события: review.rating.recalculated.

```
{
  "eventId": "uuid-or-request-id",
  "occurredAt": "2026-05-26T17:00:00.000Z",
  "source": "review-service",
  "restaurantId": "uuid",
  "avgRating": 5,
  "reviewsCount": 1
}
```

Изменения в коде

Файл	Назначение
src/common/message-bus.ts	Общее подключение к RabbitMQ, публикация и потребление событий
src/common/events.ts	Routing key и тип RestaurantRatingRecalculatedEvent
src/services/review/app.ts	Публикация события после пересчета рейтинга
src/services/catalog/app.ts	Consumer очереди и применение нового рейтинга ресторана
docker-compose.yml	PostgreSQL, RabbitMQ и сервисы приложения
scripts/smoke-test.mjs	Проверка обновления рейтинга через очередь

Проверка

Проверка выполняется командой `npm run smoke`. Smoke-тест создает отзыв, затем несколько раз читает карточку ресторана через gateway и ожидает обновления avgRating и reviewsCount. Такая проверка учитывает асинхронность RabbitMQ-сценария.

Вывод

В результате работы к микросервисной архитектуре подключен RabbitMQ. Сценарий обновления рейтинга ресторана переведен с прямого синхронного HTTP-вызова на асинхронное событие review.rating.recalculated. Это снижает связанность review-service и catalog-service и подготавливает проект к дальнейшему развитию событийной архитектуры.